

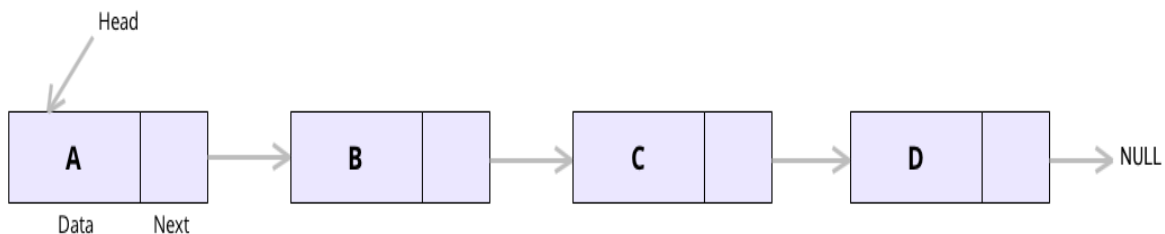
Practical # 05

Objective:

Develop a singly linked list to perform insertion, deletion, and traversal operations.

Theory:

A **singly linked list** is a dynamic linear data structure consisting of nodes. Each node contains two parts: the data part and the next part. The **data** part contains the stored data, and the **next** part provides the address of the next node. The first node of a linked list is called the **head**, and the last node is called the **tail**. The list starts traversing from the head, while the tail ends the list by pointing at NULL.



Unlike arrays, linked lists are not stored in contiguous memory locations, allowing flexible memory allocation and dynamic growth/shrinkage.

In this lab, students will learn how to:

- Create a node structure and dynamically allocate memory.
- Insert nodes at different positions (beginning, end, specific location).
- Delete nodes from different positions.
- Traverse the linked list to display its elements.

Declaring a Node type

To set up a linked list, the first thing we'll need is a structure that represents a single node in the list. For simplicity, let's assume that a node contains nothing but an integer (the node's data) plus a pointer to the next node in the list. Here's what our node structure looks like:

```
struct node{
    int value;           /* data stored in the node */
    struct node * pointer; /* pointer to the next node */
};
```

Notice that the **next member** has type **struct node ***, which means that it can store a pointer to a node structure. There's nothing about the name node, by the way; it's just an ordinary structure tag.

Now that we have the node structure declared, we'll need a way to keep track of where the list begins. In other words, we'll need a variable that always points to the first node in the list. Let's name the variable start:

```
struct node *start=NULL;
```

Setting start to null indicates that the list is initially empty.

Creating a Node

As we construct a linked list, we want to create node one by one, adding each node to the list. Creating a node requires three steps.

1. Allocate memory for the node.
2. Store data in the node.
3. Insert node in to the list.

Example program: *Declare and print elements of singly linked list.*

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node * next;
};

void print_list(Node * n) {
    while (n != NULL) {
        cout << n->data << " ";
        n = n->next;
    }
}

int main() {
    Node * head = NULL;
    Node * second = NULL;
    Node * third = NULL;

    head = new Node();
    second = new Node();
    third = new Node();

    head->data = 1;
    head->next = second;

    second->data = 2;
    second->next = third;

    third->data = 3;
    third->next = NULL;

    print_list(head);
}
```

OUTPUT

```
1 2 3
Process returned 0 (0x0)   execution time : 0.061 s
Press any key to continue.
```

Review Questions/ Exercise:

1. Implement insertion of a node at the beginning of the list.
2. Implement insertion of a node at the end of the list.
3. Develop insertion of a node at a specific position in the list.
4. Implement deletion of a node from the beginning of the list.
5. Implement deletion of a node from the end of the list.
6. Develop deletion of a node from a specific position.
7. Construct a function to count the number of nodes in the linked list.
8. Develop a program to search for an element in a singly linked list.
9. Implement a program to reverse a singly linked list.
10. Construct a program to merge two singly linked lists into one.
11. Implement a program to sort the elements of a singly linked list.
12. Develop a program to find the middle node of a singly linked list.

Name: _____

Roll #: _____

Date: _____

Subject Teacher